

LETS: A Distributed Runtime for Lineage-Bounded Autonomous Systems

System paper and artifact report for version 0.1.0

AstralDeep LETS Contributors

Standalone Apache-2.0 implementation at <https://github.com/AstralDeep/LETS>

2026-08-09

Abstract

Autonomous systems increasingly create descendants, delegates, replicas, services, and robots that act concurrently and may remain useful while disconnected. Qualitative authorization alone does not bound the aggregate quantity of effects that a recursively changing population can produce, while a central counter makes disconnected work unavailable. We present Lineage Escrow Transition Systems (LETS), a standalone distributed authorization runtime. A fixed set of stable warden processes owns disjoint shares of a finite, multi-dimensional envelope. Ephemeral subjects receive signed, expiring leases whose resources and capabilities can only be attenuated. Protected effects are state-machine transitions; a warden atomically debits a transition and emits a short-lived, audience-bound receipt that an independent executor verifies and durably claims.

This paper reports the implemented version 0.1.0 runtime rather than the earlier in-memory research kernel. It includes independent SQLite-backed warden processes, signed cluster manifests, authenticated peer HTTP, sequenced exactly-once target credit for rights transfers, branch revocation, canonical signed JSON, an authenticated HTTP API and a strict-decoding, bounded-retry client, operational tooling, and a host-neutral adapter. We give an inductive conservation argument and state the availability cost: a partitioned site may spend only rights it already owns, so safety and local progress are preserved while remote capacity can be stranded. The artifact contains 263 collected unit, integration, security, property, recovery, and end-to-end cases. A clean-volume Docker acceptance run used three distinct processes, databases, and Ed25519 keys; it exercised a bidirectional partition, transfer reordering and replay, abrupt restart, independent executor verification, and aggregate conservation. The runtime is production-intent pre-1.0 software, not a claim of Byzantine tolerance, arbitrary live migration, or generic exactly-once physical effects.

Claim discipline. IMPLEMENTED denotes behavior present in the source artifact; MEASURED denotes a reproduced observation with a recorded environment; CHECKED denotes tests, property exploration, or a bounded abstract state-space check; DESIGNED denotes a protocol argument that is not a mechanized proof; and FUTURE denotes work outside version 0.1.0. These labels prevent architecture, tests, and measurements from being reported as interchangeable forms of evidence.

1 Introduction

Agent replication is an authority-allocation problem before it is a copying problem. A child may be created by cloning code, starting a process, delegating a task, launching a container, or instantiating a robot controller. None of those mechanisms determines how much authority the resulting population can exercise. A parent that gives each child its full quota amplifies authority; a shared eventually consistent counter can overspend during a partition; and a central authorization service sacrifices disconnected progress.

LETS treats spawn as one governed transition in a more general lineage lifecycle. Its core object combines: (i) a finite integer resource vector; (ii) qualitative capabilities; (iii) an immutable transition machine; (iv) nested expiries and branch revocation; and (v) distributed escrow held by stable wardens. The system is reasoner-independent. An LLM, symbolic planner, workflow engine, or fixed controller can propose the same transition. Safety depends on complete mediation at a warden and protected executor, not on model compliance.

The component ideas are established. Database escrow partitions rights so local transactions can preserve numeric invariants (O’Neil, 1986); bounded counters extend this idea to eventually consistent systems (Balegas et al., 2015); exactly-once quantity transfer addresses duplicate delivery (Shoker et al., 2015); and finite quota can be recursively delegated (Agudo et al., 2009). Capabilities, attenuated credentials, state machines, and runtime

security automata are likewise prior art (Saltzer and Schroeder, 1975; Birgisson et al., 2014; Andersen et al., 2019; Harel, 1987; Schneider, 2000). The contribution is a narrow composition and its operational realization: stable escrow replicas are separated from churn-heavy lineage subjects, protected effects are receipts for funded state transitions, and rights move between wardens through a durable protocol rather than by copying an active subtree.

1.1 Contributions and non-claims

This artifact makes five concrete contributions.

1. IMPLEMENTED A real multi-node runtime in which each warden has an independent process, database, signing key, replay store, and initial escrow share.
2. IMPLEMENTED A durable lifecycle and transition service that provides root issuance, recursive attenuating spawn, renewal, quiescence, closure, expiry reclamation, branch revocation, and signed executor receipts under atomic local transactions.
3. IMPLEMENTED A signed, sequenced rights-transfer protocol with idempotent target acceptance, bounded reordering, acknowledgements, and bilateral prefix checkpoints.
4. DESIGNED An inductive safety argument for local and global vector conservation, capability attenuation, nested expiry, and receipt mediation, together with an explicit partition-availability trade-off.
5. CHECKED Reproducible unit, security, property, crash-recovery, bounded-model, and real three-node fault evidence, with the evidence types kept distinct.

LETS does not claim that escrow, delegation, state machines, signatures, leases, or hash-chained audit are individually novel. It does not claim Byzantine safety, automatic policy correctness, instant offline revocation, transparent active-subtree migration, or exactly-once physical effects from a receipt alone.

1.2 From research kernel to distributed runtime

The original manuscript described an in-memory Python kernel and a proposed deployment. Version 0.1.0 replaces that implementation claim with a persisted service and preserves the old kernel only as historical material. Table 1 summarizes the boundary.

Table 1: Evidence status in version 0.1.0.

Area	Artifact status	Evidence class
Local lifecycle	Durable lease, policy, receipt, audit, idempotency, and outbox transactions	IMPLEMENTED, CHECKED
Multi-node transfer	Signed prepare/accept/finalize/checkpoint protocol over authenticated HTTP	IMPLEMENTED, CHECKED
Partition behavior	Local transitions at A and B while both directed peer links are disabled	MEASURED
Restart behavior	Target killed with SIGKILL, restarted on its persisted volume, replay still denied	MEASURED
Conservation	Inductive argument, SQL commit checks, property traces, bounded abstract checker, aggregate e2e assertion	DESIGNED, CHECKED
Byzantine wardens	No threshold authorization or replicated-state-machine defense	FUTURE
Active-subtree migration	Free-right transfer followed by target issuance only	FUTURE

2 System Model and Requirements

2.1 Entities and trust boundaries

Let W be a configured, bounded set of wardens. Each warden is a stable accounting process with its own durable database and Ed25519 signing identity. Let L be the dynamic set of leases issued to agents or other subjects. Membership in L can grow and shrink recursively; membership in W changes only through an operator-controlled configuration epoch.

An envelope is named by $(tenant, envelope, epoch)$ and fixes an ordered resource schema. For m dimensions, resource values are elements of $\mathcal{R}_m = \{0, \dots, 2^{63} - 1\}^m$ with componentwise order and checked arithmetic. A policy binds that schema to one immutable state machine, lease and receipt time bounds, a clock uncertainty bound, an evidence predicate language, and a transfer gap window. Policies and machines are content addressed.

The trusted computing base consists of each warden process, its policy and configured evidence evaluators, database, clock within the configured uncertainty, signing key, manifest trust roots, protected executors, and the host isolation that prevents subjects from writing those components. Agents, generated code, prompts, memory, discovery documents, request bodies, networks, and other tenants are untrusted. Wardens are trusted but may crash. Messages may be lost, duplicated, delayed, reordered, replayed, or partitioned.

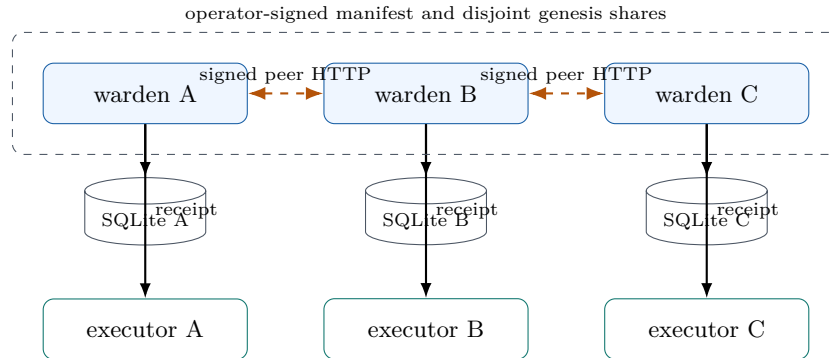


Figure 1: Reference topology. The Compose acceptance deployment instantiates three independent warden processes, stores, and keys; its test exercises one executor with a replay store outside the warden database. Protected domains may deploy similarly isolated executors per site. Ephemeral subjects call wardens but are not escrow replicas.

2.2 Safety and availability requirements

The principal safety property is that no execution schedule creates resource rights. Spawn and transfer may relocate rights; a transition may move them into consumed accounting; reclamation may return unconsumed rights; no operation may increase the global envelope. Capability and machine authority must attenuate down a lineage, lease expiry must be nested, and an executor must reject an invalid, stale, wrongly addressed, or replayed receipt.

Availability is conditional. A healthy warden should authorize from its local share without a peer round trip. Under a partition, a site can continue only while it has locally escrowed rights, valid leases, a sufficiently certain clock, and a reachable protected executor. Remote rights are not borrowed speculatively. This chooses bounded authority over full utilization during every partition.

3 LETS Design

3.1 Local ownership equation

For warden w , let $\mathbf{S}_w$ be its signed genesis share, $\mathbf{I}_w$ cumulative accepted inbound transfer, $\mathbf{F}_w$ its free pool, $\mathbf{R}_w$ the sum of residual rights in unreclaimed leases, $\mathbf{C}_w$ cumulative consumption, and $\mathbf{O}_w$ cumulative prepared outbound transfer. The persisted local invariant is

$$\mathbf{S}_w + \mathbf{I}_w = \mathbf{F}_w + \mathbf{R}_w + \mathbf{C}_w + \mathbf{O}_w. \quad (1)$$

Every term is a nonnegative vector of the same dimension. The manifest verifies $\sum_w \mathbf{S}_w = \mathbf{B}$, where $\mathbf{B}$ is the global initial budget. Since an inbound credit can exist only for a signed, already prepared source voucher, define the source-owned in-flight vector on a consistent cut as $\mathbf{X} = \sum_w (\mathbf{O}_w - \mathbf{I}_w)$. Summing Eq. 1 yields

$$\mathbf{B} = \sum_w (\mathbf{F}_w + \mathbf{R}_w + \mathbf{C}_w) + \mathbf{X}. \quad (2)$$

Before target acceptance, transferred rights are in $\mathbf{X}$; after acceptance, matching outbound and inbound cumulative terms cancel. This formulation avoids counting a prepared voucher as both source capacity and target capacity.

3.2 Signed leases and recursive attenuation

A lease grant binds tenant, envelope, epoch, lease and lineage identifiers, parent and subject, original allocation, capabilities, policy and machine digests (the latter committing to the initial machine state), ancestor path, branch epoch, issue and expiry times, warden and key identifiers, and a signature. The mutable snapshot contains residual vector, current state, lifecycle status, and monotonically increasing sequence.

Root issuance is a privileged operation: a transport-authenticated identity must hold `lets.admin`, `lets.warden.admin`, or `lets.lease.issue`. It debits the local free pool and cannot request a capability absent from the bound machine; the allocation must contain at least one positive component. Spawn is available to the authenticated parent subject or an identity holding `lets.admin` or `lets.lease.manage`. A child allocation is likewise nonzero. For parent p and child c , the transaction requires

$$\begin{aligned} \mathbf{a}_c &\leq \mathbf{r}_p, & K_c &\subseteq K_p, & (3) \\ t_c^{exp} &\leq t_p^{exp}, & d_c^{policy} &= d_p^{policy}, & d_c^{machine} &= d_p^{machine}. & (4) \end{aligned}$$

It then atomically replaces $\mathbf{r}_p$ by $\mathbf{r}_p - \mathbf{a}_c$ and creates the child with residual $\mathbf{a}_c$. A request cannot substitute a cheaper or more permissive machine behind a different policy identifier. Expected state and sequence fields provide optimistic concurrency guards. A durable request identifier is unique envelope-wide across mutation types; its full fingerprint makes retries idempotent and rejects a same-key/different-operation or body collision.

Replication has deliberately narrow semantics. It creates a new governed runtime identity and partitions authority. It does not copy bearer tokens, private keys, human identity, process memory, open sockets, uncommitted state, protected data, or signing secrets. Artifact and workspace transfer remain host operations.

3.3 Transition authorization and protected execution

Each protected transition specifies source state, target state, required capability, a componentwise nonnegative resource cost with at least one positive component, and an optional declarative evidence rule. The evidence language is data, not executable policy code; policy and machine digests include its semantics. Evidence can satisfy a guard but cannot mint resources or capabilities.

After the network boundary authenticates the request and the service validates the identity object, authorization performs the following stateful checks and mutations in one local write transaction:

1. tenant/subject/scope binding; lease status, branch epoch, expiry, and clock checks;
2. request-id fingerprint lookup and optional expected state/sequence checks;
3. immutable policy/machine resolution, legal transition, capability, audience, and evidence evaluation;
4. componentwise residual debit and machine state/sequence update;
5. signed receipt, idempotent response, hash-chained audit event, and outbox persistence; and
6. durable commit before returning the receipt.

The protected executor has a separate trust configuration and replay database. It always verifies the exact audience, receipt freshness under bounded clock uncertainty, and signature against a registered warden/key pair that is currently valid at verification time. When local `ExecutorPolicy` fields or allowlists are configured, it additionally pins tenant, envelope, epoch, policy digest, machine digest, and warden. Its durable claim then requires the lease sequence to advance and claims both receipt identifier and nonce. This supplies at-most-once authorization consumption. A generic receipt cannot make an arbitrary physical effect exactly once: the executor must bind receipt claim and effect in one domain transaction, make the effect idempotent, or use compensation.

3.4 Lifecycle and revocation

Quiescence disables transition execution without inventing a second policy-machine state. Resume restores actionability if the lease remains valid. Renewal is bounded by the policy maximum and by the parent expiry; a root renewal additionally requires `lets.lease.manage` or `lets.admin`. Version 0.1.0 refuses parent shortening that would strand a live longer-lived descendant. Close and expiry reclamation return only the lease's own residual, once. Reclamation waits past clock uncertainty and the maximum outstanding receipt lifetime so a receipt cannot remain valid after its authority has been reassigned.

A branch revocation is a signed monotonic epoch for a lineage/branch prefix. Only the branch-owning warden may create it; authenticated peers may relay the owner-signed record. Connected descendants fail closed once the revocation is visible. A disconnected descendant cannot be revoked instantly without communication. Its additional

resource exposure is nevertheless bounded componentwise by the residual already present in that branch, and its time exposure is bounded by nested lease expiry plus the configured clock/receipt safety margin under complete mediation. This is an engineering bound, not a claim that a compromised executor or bypass path is contained.

4 Cross-Warden Rights Transfer

4.1 Protocol

For each $(source, target, envelope)$ stream, the source allocates strictly increasing sequence numbers. The protocol is a durable handoff, not a distributed transaction spanning both databases.

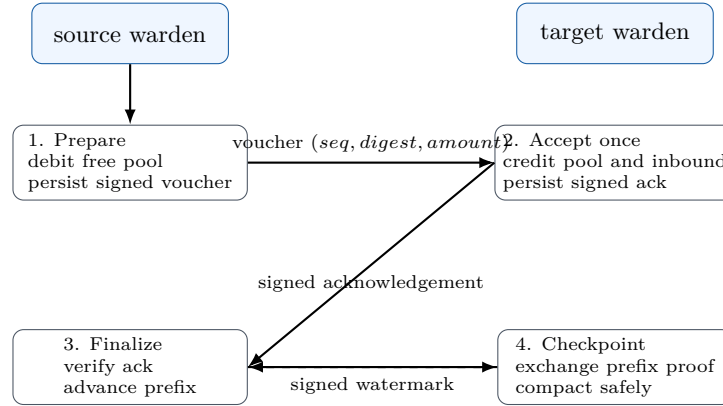


Figure 2: Transfer ownership moves monotonically. A timeout never refunds a prepared source voucher; retries recover the acknowledgement without a second target credit.

Prepare. The source atomically subtracts the amount from its free pool, increments cumulative outbound accounting, assigns the next sequence, and persists a signed voucher. A timeout does not restore the amount. Before target acceptance it remains source-owned in-flight authority in Eq. 2, but it is no longer spendable.

Accept. The target authenticates both the peer HTTP message and the signed voucher, checks tenant/envelope/epoch/source/target/policy binding, and enforces a bounded gap window. Unique sequence and transfer-id bindings prevent alternate payloads. The first valid acceptance credits the free pool and cumulative inbound accounting in one transaction and stores a signed acknowledgement. A business-level duplicate returns the stored logical acknowledgement without a second credit; an exact replay of the signed HTTP envelope is rejected by the durable nonce store.

Finalize and checkpoint. The source verifies the target signature and voucher digest, records finalization idempotently, and advances a contiguous acknowledgement watermark. Once both peers agree on a finalized contiguous prefix, a signed checkpoint permits deletion of individual rows below that prefix. Gaps, conflicting identifiers, backward watermarks, unknown epochs, and untrusted keys fail closed.

IMPLEMENTED A running warden’s bounded peer dispatcher discovers authoritative transfer, revocation, and checkpoint records through durable delivery metadata. It schedules one head per ordered stream, signs HTTP requests, retries failed delivery with bounded exponential delay, finalizes acknowledged transfers, creates checkpoints, and prunes only terminal delivery metadata. Work is not lost if the process crashes between the authority transaction and dispatcher discovery. At-least-once peer delivery remains conditional on connectivity, a live trusted peer, and both signing keys remaining current; idempotent receiving converts redelivery into exactly-once accounting credit, not exactly-once network transmission.

4.2 Failure outcomes

Table 2 states the intended result, including the availability cost.

Table 2: Transfer and partition outcomes. “Safe” means no duplicated spendable rights under the trusted-warden model; it does not imply liveness.

Failure	Result	Property
Voucher lost after prepare	Amount stays unspendable at source; retry same durable voucher	Safe, unavailable
Acknowledgement lost	Redelivery returns stored acknowledgement; target is not credited twice	Safe, retryable
Out-of-order delivery within window	Target records gap; contiguous watermark advances when missing prefix arrives	Safe, bounded memory
Gap beyond window or conflicting sequence	Request rejected	Fail closed
Target crash after commit	Persisted credit, acknowledgement, and replay state recover on reopen	Safe after recovery
Peer partition	Each warden continues only from local rights; transfer stalls	Safe, partial progress
Signer reaches finite not_after with pending records	Verification continues to fail closed; prepared rights or revocation/checkpoint delivery can remain stranded	Safe, unavailable
Cloned source database and key run concurrently	Outside version 0.1.0 model; can duplicate authority	Unsafe/non-goal

5 Implementation

The implementation is a typed Python package supporting Python 3.11–3.14 ([AstralDeep, 2026](#)). The core depends on PyNaCl 1.6.2 over libsodium for Ed25519; optional server and client extras provide FastAPI/Uvicorn and HTTPX. Development dependencies are resolved by the committed lockfile into a repository-local .venv. Containers build an image-local environment and do not mount the host environment.

5.1 Durable storage and atomicity

Each SQLite database owns one envelope. It uses foreign keys, write-ahead logging, synchronous=FULL, a bounded busy timeout, schema/application identifiers, migrations, startup integrity checks, and `BEGIN IMMEDIATE` for serialized writes ([SQLite Project, 2026](#)). Checked vector functions are registered with SQLite. Triggers maintain the lease-residual aggregate and validate dimensions; a transaction-level pre-commit check rejects a local conservation violation. Compound foreign keys prevent mixing a policy version with another digest. The authoritative tables cover metadata, warden state, immutable policies, leases, receipts, idempotency responses, revocations, inbound and outbound transfer streams, checkpoints, signed audit, and outbox state.

The storage abstraction admits another backend, but only SQLite has equivalent implementation and fault evidence in this release. The release dependency audit selected PyNaCl/libsodium for the signed path and removed an unused PostgreSQL extra; version 0.1.0 exposes neither a PostgreSQL dependency nor a validated adapter.

5.2 Canonical signed data and key trust

All content digests and signatures use the documented LETS-CJ/1 profile. Object keys are sorted by Unicode scalar value, arrays retain order, integer values are lossless signed 64-bit values, and output is minimal UTF-8 JSON. Floats, non-finite values, duplicate keys, non-string keys, lone surrogates, and non-canonical base64url spellings are rejected. LETS-CJ/1 is intentionally not RFC 8785 JCS because nanosecond timestamps and resource values may exceed the lossless integer range of common IEEE-754 JSON consumers ([Rundgren et al., 2020](#)). Cross-language implementations must use the published conformance vectors and lossless parsing.

Signed-record type tags use the LETS-owned `lets.*` version namespace; the cluster manifest uses `lets.manifest/v1`. Neither the core protocol nor its canonical bytes contain an AstralDeep-specific type namespace.

Ed25519 authority records bind both warden and key identifiers. The trust registry rejects conflicting registrations. Manifest-bound service startup, readiness, and every authority-record signing path require the complete

declared clock-uncertainty interval to fit within the signer’s configured not-before/not-after interval; peer receivers independently enforce current-time key validity for protocol records and signed HTTP requests. A new policy registration applies the authority-signing rule before commit. The cluster manifest binds the envelope schema, global budget, exact disjoint shares, peer endpoints, keys, and policies; local bootstrap trust configuration fixes the accepted operator keys and required signature threshold. Manifest verification counts distinct operator key material rather than aliases, requires warden key identifiers and Ed25519 public-key material to be globally unique, and forbids Ed25519 public-key material reuse across operator and online warden roles. Non-loopback plain HTTP endpoints require an explicit development override; production deployment requires TLS in addition to message signatures.

The warden database never stores the private signing seed. Its immutable bootstrap metadata binds the initial warden/key identifier and SHA-256 public-key fingerprint; signer substitution therefore fails startup. Manifest-backed service startup reloads strict JSON and re-verifies the manifest digest, operator threshold, local key/configuration, and complete peer trust before serving.

Version 1 resource limits bound adversarial parser and validation work: at most 256 resource dimensions, 1,024 machine transitions, 64 ancestor-path entries, and evidence expression depth 32 with at most 256 nodes and 256 membership values per node. These are protocol limits, not measured capacity guidance.

5.3 Network planes

The client plane strictly decodes JSON and rejects unknown top-level fields in endpoint request bodies; deliberately opaque evidence and configuration values remain open, and dynamic policy registration performs targeted semantic checks but does not enforce the complete nested OpenAPI Policy schema. The published OpenAPI 3.1 artifact documents closed request schemas for clients, endpoint 2xx response schemas, and application/problem+json errors. Runtime handlers produce those responses, but the bundled client does not validate success bodies against OpenAPI. Identity is derived by an injected authenticator, never accepted from body fields. The bundled static bearer authenticator is bootstrap tooling and stores only token digests. Production deployments should inject an OIDC, mTLS, gateway, or workload-identity verifier.

Authentication is enforced at the network boundary. Direct calls to service and administrative ports are trusted in-process integration calls, not authenticated remote entry points; an embedding host must keep them inside its TCB.

Peer requests carry method, raw target path/query, body digest, timestamp, nonce, warden, key, and an Ed25519 signature. A dedicated SQLite replay store atomically prunes expired entries and claims new nonces. Peer signatures authenticate content but do not provide confidentiality or hide metadata; TLS remains mandatory outside explicit development topology.

The operator surface provides liveness, clock-aware readiness, invariant snapshots, audit verification, bounded audit pagination, and metrics for pools, consumption, transfer watermarks/gaps, and outbox lag. A verified SQLite backup command refuses an existing destination and validates the produced snapshot. Restore is operationally safe only after fencing the original node.

5.4 Module decomposition

Table 3: Major implementation boundaries.

Module	Responsibility
policy, models	Immutable transition/evidence semantics and strict signed wire records
service	Lifecycle, transition, revocation, transfer, invariant, and audit transactions
storage.sqlite	Versioned schema, durability, repositories, and transactional conservation checks
crypto, canonical	Ed25519 trust and LETS-CJ/1 bytes/digests
auth, api, client, peer	Transport identity, signed HTTP/replay defense, durable peer dispatch, and client/CLI contracts
executor	Independent receipt verification and durable at-most-once claim
manifest	Operator-signed cluster bootstrap and exact genesis-share validation
integrations	Host-neutral lifecycle port and optional AstralDeep scope profile

6 Safety Argument

This section is a protocol-level inductive argument, not a machine-checked refinement proof. The bounded checker and executable tests discussed in Section 7 provide separate, finite evidence.

6.1 Local conservation

Lemma 1 (local preservation). If Eq. 1 holds before a committed operation, each implemented operation preserves it componentwise.

Argument. Root issuance moves a vector from F_w to R_w . Spawn moves a vector between parent and child terms inside R_w . An authorized transition moves its cost from R_w to C_w . Close or reclamation moves a residual from R_w to F_w . Quiescence, resume, renewal, and revocation change status or time metadata but not resource terms. Prepare subtracts from F_w and adds to O_w . Target acceptance adds the same voucher amount to both F_w and I_w . Finalization and checkpointing change no vector. Retried idempotent operations return the prior result; rejected or rolled-back operations change no authoritative state. Checked arithmetic prevents wraparound. $\square$

Theorem 1 (global conservation under the failure model). If the signed manifest exactly partitions B , all wardens begin in a locally valid state, peer keys are trusted, and every committed operation is one of the operations above, then Eq. 2 holds on every consistent cut despite crash, retry, duplication, reordering, loss, or partition.

Argument. Lemma 1 preserves every local equality. A target increments I only after verifying a matching signed prepared voucher; uniqueness constraints prevent a second credit. Therefore cumulative inbound never creates an unmatched source debit. Summation and the manifest share equation yield Eq. 2. Message failures affect whether a prepared term moves out of X , not whether it is duplicated. $\square$

The theorem excludes a compromised warden signing nonexistent vouchers, a simultaneously running database/key clone, manifest trust-root compromise, and direct database modification.

6.2 Lineage and transition properties

Lemma 2 (attenuation). Along every issued lineage edge $p \rightarrow c$, the child’s capability set is a subset of the parent’s, the immutable policy/machine digests are equal, the child expiry is no later than the parent’s, and the child allocation is removed from the parent in the same transaction.

Argument. These are explicit preconditions and writes of the spawn transaction. Subject binding prevents an unscoped sibling from invoking spawn on another lease. The policy digest and machine digest are signed into the grant. $\square$

Lemma 3 (transition soundness). Under complete mediation and uncompromised trusted code, an executor accepts a receipt only for a committed transition whose source state, capability, evidence, vector cost, policy/machine digests, audience, sequence, and validity interval passed the warden checks.

Argument. The debit, state/sequence update, signed receipt, idempotency response, and audit record commit atomically. The executor independently verifies all signed bindings and atomically claims the receipt. A missing, invalid, stale, or replayed receipt fails closed. $\square$

Lemma 3 says nothing about the semantic adequacy of a configured cost or evidence predicate, nor about effects reachable through a bypassable executor.

7 Evaluation and Reproducible Evidence

The evaluation asks whether the implementation satisfies its protocol obligations under bounded and adversarial schedules. It is not a claim of external security certification or broad workload performance.

7.1 Evidence layers

7.2 Bounded formal exploration

The artifact supplies a finite TLA+ specification of scalar escrow, transfer, receipt, and replay semantics (Lamport, 1994). TLC completed its configured TypeOK and conservation checks with 40,057 generated states, 6,776 distinct states, maximum depth 14, and an empty queue. A separate dependency-free executable abstract model explores root issuance, recursive spawn, consumption, close/reclamation, sequenced transfer preparation, reordering, acceptance,

Table 4: Verification layers and what they can establish.

Layer	Reproduced status	Scope and limitation
Static checks	Ruff check/format, strict mypy on 24 source files, and uv lock check passed	Ruff formatting/lint and strict mypy check source consistency; neither proves protocol safety.
Test suite	262 passed; 1 e2e deselected; 75.46% branch-aware coverage; 14.91 seconds	Unit, integration, security, property, concurrency, crash, and recovery behavior; finite executions only.
Compatibility/package	Python 3.11.15: 262 passed, 1 deselected, 75.44% branch coverage; 23.44 seconds; wheel/sdist and isolated import checks passed; Bandit and pip-audit clean	One additional interpreter and isolated artifact/import checks; not a platform certification.
TLA+ TLC	passed; 6,776 distinct states	Scalar three-warden escrow/transfer/receipt kernel; finite and not a source-code refinement.
Bounded checker	passed; 101,245 states	scalar budget 3; three wardens; at most three leases, two transfers, and two receipts; unit actions; depth 9; abstracts signatures, clocks, SQL, and HSM labels.
Docker acceptance	1 passed in 6.47 seconds	One composite clean-volume fault scenario on one Windows/Docker host; not a throughput or multi-host WAN study.
Microbenchmarks	100 operations/path; FULL durability	Development measurements only; no cross-system baseline unless reported with environment and repetitions.

finalization, receipt claim, and duplicate delivery. Under scalar budget 3; three wardens; at most three leases, two transfers, and two receipts; unit actions; depth 9, it explored 101,245 states, 318,558 transitions, and 66,072 idempotent self-loops; zero invariant violations. A fault-injection mode that credits an already accepted transfer found a three-action prepare, accept, duplicate-accept counterexample at depth 2. The latter is a checker-sensitivity test, not evidence that the production implementation contains that fault. Both models abstract signatures, clocks, SQL, HTTP, and key labels; neither is an unbounded proof or a refinement check against the runtime. Executable service trace conformance is tested, not proved as a refinement mapping.

Property tests independently generate checked-vector inputs and randomized lifecycle traces. Security cases target privilege boundaries, policy substitution, request-id collision, evidence type confusion, concurrent parent overdraw, transfer identity/sequence conflicts, signature spelling, peer key validity, executor races, and canonical wire parsing. Recovery cases open a database after rollback, committed transaction, and abrupt process exit.

The release gate ran the non-e2e suite with ResourceWarning promoted to an error and branch-aware coverage enabled: 262 passed; 1 e2e deselected; 75.46% branch-aware coverage; 14.91 seconds, against a configured 74% floor. The real Docker e2e case is reported separately below so an unavailable Docker daemon cannot be hidden inside the core-suite result. A separate exact all-extras environment on Python 3.11.15 produced Python 3.11.15: 262 passed, 1 deselected, 75.44% branch coverage; 23.44 seconds. The package matrix built and inspected the wheel and source distribution, smoke-tested base/client/server imports on Python 3.14, and completed Twine, wheel-content, Bandit, and pip-audit checks without a finding; these are bounded tool results, not an external security certification. The wheel was 129,808 bytes with digest `sha256:38491a30929a2cf81c4c4ec3af75b677f441742aa8a9f8c5c22107a7e0a418e9`; the source distribution was 118,026 bytes with digest `sha256:20fa4c5dab416e0e00f0be77f739ea6cfea3d20b2e2250baddafdb44ec4456b3`.

7.3 Post-optimization storage measurements

MEASURED The reviewed baseline used Windows 10 build 19045; Python 3.14.0; SQLite 3.50.4; AMD64; 32 logical CPUs and the release PyNaCl/libsodium backend. Each main path ran 10 warm-up and 100 measured operations with `perf_counter_ns` and nearest-rank percentiles. Every writing path kept SQLite WAL synchronous=FULL and one atomic transaction per stage; verify-only has no write. The worktree was uncommitted, and the single-host in-process measurements exclude HTTP, network, container scheduling, and independent failure domains. The reviewed machine-readable baseline is retained under `benchmarks/baselines`.

Table 5: Post-optimization latency and wall-time rate. The 100-sample p95 values are indicative, not capacity guarantees.

Path	Median (ms)	p95 (ms)	Rate (op/s)
Warden authorize	7.728	8.588	128.73
Transfer prepare	6.902	7.828	139.28
Transfer accept	6.267	7.190	154.86
Transfer finalize	5.721	6.547	173.97
Transfer round trip	18.855	21.190	51.59
Executor verify only	0.177	0.183	5,586.94
Executor verify and durable claim	3.830	4.740	252.38
Four-worker authorize contention	9.795	80.881	119.69

Authorization includes validation, evidence/policy evaluation, Ed25519 signing, audit, invariant maintenance, and durable commit. The transfer round trip is three FULL transactions across two stores and excludes HTTP/network time. The four-worker row uses independent leases in one single-writer database; latency includes lock wait, and its 80.881-ms p95 shows why the result must not be generalized to arbitrary write concurrency.

IMPLEMENTED The final optimization pass removed an unconditional full `PRAGMA foreign_key_check` from every successful write while retaining per-connection foreign key enforcement, deferred-constraint rejection at commit, and full startup/explicit diagnostics. MEASURED The isolated diagnostic median grew from 0.01410 ms at 10 receipt rows to 1.87145 ms at 1,000 rows (approximately 133-fold). A paired 1,000-row authorize result fell 17.8%, but security hardening and the cryptographic-backend change occurred between runs, so the paper does not attribute that full delta to the foreign-key optimization.

IMPLEMENTED Frequent invariant snapshots now read a trigger-maintained lease-residual aggregate while startup and explicit diagnostics retain full reconciliation. At 10, 100, and 1,000 lease rows, the legacy row-scan medians were 1.070, 1.295, and 2.902 ms; aggregate medians were 1.112, 1.090, and 1.085 ms. The 1,000-row observation was 62.6% lower and the aggregate medians were flat over this bounded range. The legacy observations were taken immediately before the edit on the same host but predate the reusable harness.

A read-only keeper-connection experiment was rejected: four order-alternated trials gave 6.0969 ms without the keeper and 7.2333 ms with it, a 18.64% penalty. The release therefore retains per-operation connections. The final profile was dominated by the durable SQLite connection/commit lifecycle; no further source change was justified by this bounded measurement without weakening transaction or durability semantics. The fresh exact dependency graph contains 48 packages in a 356,264-byte lockfile; the PyNaCl/libsodium backend and its CFFI support occupy 1,778,149 installed bytes in that environment. The lock contains no cryptography package.

7.4 Real three-node fault acceptance

The clean-volume Compose run used Python 3.14.7, Docker Engine 29.6.2, and Docker Compose 5.3.1. It ran three warden containers with distinct process IDs, independent SQLite volumes, and three distinct Ed25519 keys from one operator-signed manifest. Initial shares 150 + 100 + 50 exactly partitioned a 300-unit envelope.

The acceptance runner rejected any source change across the run. Its explicit, non-circular 46-file runtime/acceptance input set had digest `sha256:8216a535323863ca1d9ff523adfb29597ac7d02fe2e871985c4851433df13e56`; the complete 123-file Git-visible snapshot had digest `sha256:b3402e15d92a1b0d0cccec2aac1ff488a6cdd44e944c4a1ef14df8164b434ac5e`. The rebuilt runtime image content identifier was `sha256:fe174161832f624c01a005fa1253607356a194a78d669eb72aec4eec3af52ef6`. Paper prose and derived evidence are excluded from the runtime-input set, so this manuscript rebuild does not invalidate the recorded execution.

The scenario delivered transfer sequence 2 before sequence 1. The target held its contiguous watermark at 0 and returned stable acknowledgements for duplicate application delivery. The harness disabled both directed A-B proxy links: peer delivery failed while its two vouchers remained durable and A and B each committed a local authorization receipt. It then killed warden B with SIGKILL; the restarted process had a new PID and the same persisted key. An identical signed HTTP request was rejected with status 409 and code `replay_detected` both before and after restart. After link restoration, the production dispatcher automatically accepted and finalized both transfers, delivered the bilateral checkpoint, advanced both compacted high-water marks to 2, and converged to zero pending delivery records and zero prepared transfers.

An executor outside the warden database verified one receipt, claimed it in its own SQLite replay store, reopened that store, and rejected the same receipt. At the final consistent cut, cumulative inbound and outbound transfers were both 20, the sum of free pool, lease residual, and consumed rights across nodes was 300, every local invariant reported healthy, and all three signed audit chains verified. The pytest result was 1 passed in 6.47 seconds; the full provenance-bound runner took 23.782 seconds. The manifest digest and machine-readable scenario are retained in the artifact evidence record; the run’s manifest digest was `sha256:1c3f47302300af39e50b065327d307a46e883c5d1e4f44d3de37c7bda4f3250f`.

This scenario establishes that the hard acceptance path is not an in-process simulation. It does not establish geographic independence, behavior under arbitrary schedules, long-duration stability, or Byzantine tolerance.

8 Interoperability Profiles

LETS is an authorization/accounting substrate, not an agent discovery protocol, invocation transport, artifact registry, identity provider, or telemetry backend. Integration therefore uses explicit profiles and keeps discovery separate from authority.

Table 6: Standards and draft composition. “Profile” means a mapping boundary, not bundled conformance.

Concern	Existing standards and drafts	LETS relationship
Discovery and description	A2A Agent Cards (A2A Project, 2026), OASF (AGNTCY Contributors, 2026), Agent Spec (Amini et al., 2025), ARD v0.9 draft (ARD Project, 2026)	Map immutable IDs, advertised skills, endpoints, and artifact references into host policy inputs. Advertisement never grants a lease.
Invocation	MCP 2026-07-28 (Model Context Protocol Maintainers, 2026), A2A tasks/messages (A2A Project, 2026)	Carry a host operation to the adapter. A protected effect still requires a LETS receipt at the executor.
Workload identity	SPIFFE 1.15.2 (SPIFFE Project, 2026), OIDC or mTLS	Inject verified transport identity and key trust. The release includes an authenticator interface, not a bundled SPIFFE control plane.
Artifact and build provenance	OCI (Open Container Initiative, 2025), SLSA 1.2 (SLSA Community, 2025), in-toto 1.2 (in-toto Project, 2026)	Host stores immutable artifact descriptors and attestations; LETS binds their identifiers or evidence digests where policy requires.
Composition and inventory	SPDX 3.0.1 (SPDX Workgroup, 2024), CycloneDX 1.7 (OWASP Foundation, 2025)	Supply machine-readable SBOM/AI-BOM evidence; LETS does not invent another BOM.
Provenance and events	W3C PROV-O (Lebo et al., 2013), CloudEvents 1.0.2 (CloudEvents Authors, 2022)	Export audit/outbox observations for analysis. Exported events are not authority and cannot repair a warden pool.

The host-neutral ReplicaAuthorizer accepts lifecycle IDs, resource vectors, capabilities, TTLs, evidence, and immutable policy references. It deliberately excludes credentials, process memory, owner identity, sockets, and opaque agent state. A profile capability set is an upper bound; unknown mappings fail closed.

AstralDeep is one optional consumer, not a dependency of the core. The included adapter maps only the six declared AstralDeep tool scopes to explicitly configured LETS capabilities and transitions. It imports no AstralDeep module and writes no AstralDeep table. AstralDeep retains its Keycloak/RFC 8693 identity path, owner isolation, permissions, PHI and egress checks, confirmations, and audit. This same separation supports non-agent domains such as robotics, logistics, infrastructure, healthcare workflows, manufacturing, or economic limits: only resource dimensions, machines, evidence rules, and adapter mappings change.

9 Security Analysis

9.1 Attacks addressed

The runtime rejects unscoped self-issuance from the free pool, cross-tenant operations, child capability amplification, policy/machine substitution, residual overdraw, stale expected sequences, duplicate-key JSON, floating-point signed data, alternate base64url spellings, conflicting transfer IDs or sequences, target/source substitution, expired keys, excessive clock uncertainty, wrong receipt audience, and durable nonce/receipt replay. Warden, executor, and peer replay stores persist nondecreasing observed-time floors and reject wall-clock rollback beyond the configured tolerance or declared uncertainty, including after reopen. A large forward clock jump can instead expire authority early and reduce availability. Concurrency tests exercise one-winner behavior for parent debit, command idempotency, and executor claim.

Hash-chained signed audit makes row modification or deletion detectable when the complete chain and trusted key are available. It does not provide an external anti-rollback anchor: restoring an older database and key together can present an internally valid old prefix. Operators should export and anchor checkpoints outside the node when rollback detection matters.

9.2 Residual risks and explicit boundaries

Table 7: Version 0.1.0 boundaries.

Boundary	Consequence and required control
Trusted wardens	A compromised warden can sign invalid authority from its assigned share. Use host isolation and protect keys; threshold/Byzantine designs are future work.
One envelope per SQLite DB	Scale by sharding independent envelopes across processes. A PostgreSQL backend requires its own conformance and fault evidence.
Restore clones	Never run original and restored copies of one database/key concurrently. Fence the original before restore.
Clock dependence	Expiry and key validity rely on bounded uncertainty. Excessive uncertainty fails closed and can reduce availability.
Clock rollback	Durable time floors block ordinary rollback but not a cloned or operator-rewritten database. Large forward jumps can expire authority early.
No live epoch rollover	Drain into a new envelope/epoch; automated in-place membership and key rotation are not present. A finite online-key expiry is a hard drain deadline.
No active-subtree migration	Transfer free rights, then issue at the target. Do not copy a live lease as authority.
No generic effect transaction	Receipt claim is at-most-once. Bind it to an idempotent or transactional domain effect.
Evidence semantics	A valid signed sensor report can still be false or semantically insufficient. Domain trust anchors and policy review remain necessary.
Privacy	Lease paths and audit records reveal organizational activity. Minimize fields and protect/export them under domain retention policy.
Plaintext state	SQLite contains sensitive authority, lineage, and audit metadata. POSIX mode 0600 is best effort; Windows/container ACLs, volume encryption, and backup protection are operator controls.
Embedding boundary	Direct in-process service/admin calls are trusted ports. Expose only the authenticated API or reproduce its identity/scope checks.

Finite manifest key intervals are availability boundaries. Version 1 evaluates key validity at verification time and does not require lease or receipt expiry, or the open-ended retry horizon of peer records, to fit before `not_after`. Once the declared clock interval reaches that boundary, local readiness and every authority-record signing path fail closed; verifiers also reject otherwise-unexpired receipts and undelivered transfer, revocation, or checkpoint records. Active version 1 online warden keys should therefore normally omit finite `not_after`. If one is configured, treat it as a hard drain deadline: before it, stop new authority work, close or reclaim live leases, wait out outstanding receipts, prove all durable transfer and peer-delivery state terminal, and fence the old epoch before starting a separately initialized replacement epoch. Automated in-place key rotation is not implemented. The included raw seed-file signer is portable bootstrap machinery; production systems should prefer an external or hardware-backed key provider.

10 Related Work and Novelty Boundary

Agent Contracts combines multidimensional budgets, lifecycle semantics, recursive delegation, and conservation (Ye and Tan, 2026); Agent libOS provides identity, parent-child lineage, capabilities, checkpoints, and audit for long-running agents (Zhang, 2026). These works invalidate broad novelty claims for hierarchical agent contracts or secure agent runtimes. LETS instead targets the distributed operational semantics of a population-wide hard effect vector during partitions, with stable wardens rather than every descendant serving as a counter replica.

Provenact studies policy-state serializability for concurrent effects (Peng and Wu, 2026). Fixed Ceiling studies authorization continuity and attenuating authority across agent evolution (Zhang and Zhang, 2026). Faramesh places a protocol-neutral authorization boundary before execution (Fatmi, 2026), while AgentBound issues verifiable governance receipts bound to delegation and behavioral artifacts (Kaul et al., 2026). LETS shares their emphasis on execution-time mediation and verifiable decisions; its narrower distinction is conserved, partitioned quantity across concurrently acting lineage branches and cross-warden handoff.

Empirical work such as Agent Matrix motivates treating replication as a concrete systems risk rather than a purely hypothetical capability (Zhang et al., 2025). LETS does not try to detect whether a model “intends” to replicate. It makes spawn an ordinary authorized operation that cannot create net resource authority. This turns a model-behavior question into a mediation and accounting obligation, conditional on complete coverage of protected effects.

The stable-warden design inherits the utilization trade-offs of escrow and bounded counters (O’Neil, 1986; Balegas et al., 2015). Rights placed at an isolated site cannot be consumed at another site until connectivity returns. Dynamic rebalancing, predictive placement, and broader WAN evaluation are optimization questions layered above the invariant, not reasons to relax it.

11 Operational Guidance

A production deployment starts from one operator-reviewed manifest that exactly partitions every budget dimension and binds all initial keys, endpoints, policies, and epoch metadata. Each private key, database, replay database, configuration, and manifest trust root form one authority unit. Production endpoints use TLS; the cleartext Compose topology and public default token are fault-test fixtures only.

Operators should monitor readiness, disk and WAL growth, write latency and busy errors, clock uncertainty, lease and receipt expiry, transfer gaps and watermarks, outbox lag, invariant status, and audit verification. A source voucher is never manually refunded after timeout. Backups are offline or coordinated snapshots; restores are reconciled against peer watermarks and audit checkpoints before returning to readiness. Idempotency and signed audit history are not deleted by hand.

The single-writer SQLite design is appropriate for an independently owned envelope with bounded write concurrency, not an assertion that one process should serve arbitrary fleet scale. Scale-out uses more independent envelopes today. A different store must reproduce transaction isolation, vector checks, uniqueness, recovery, replay, and transfer tests before it can replace SQLite in the trusted path.

12 Limitations and Future Work

The principal internal-validity threat is the gap between protocol arguments and executable refinement. The artifact includes a finite TLA+ kernel and a separately executed abstract checker, but there is no machine-checked refinement connecting every SQL transaction and crash point to Eq. 2. The bounded checker covers only one small finite state space. Independent audit and multi-language canonicalization interoperability are still required.

The reported Docker run is one composite scenario on one host. It verifies independent operating system processes and storage volumes, but not independent failure domains, heterogeneous architectures, real WAN clocks, sustained partitions, rolling upgrades, or long-duration load. Future evaluation should add repeated seeded fault campaigns, protocol-version skew, disk-full and I/O failure, clock excursions, TLS and external identity profiles, and multi-host latency and throughput distributions.

Engineering priorities are threshold authorization for compromised-warden tolerance, safe configuration-epoch rollover, externally anchored audit/checkpoints, a validated PostgreSQL adapter, hardware-backed keys, policy analysis, and workload-informed rights placement. Active subtree migration requires a separate handoff protocol that proves a single owner for every live lease and outstanding receipt; it must not be approximated by copy-and-delete.

13 Conclusion

LETS version 0.1.0 is a real distributed, multi-node runtime for governing recursively changing populations under finite effect envelopes. It separates stable escrow wardens from ephemeral subjects, partitions rights rather than duplicating them, authorizes funded state transitions atomically, and requires independently verifiable receipts at protected executors. Cross-warden transfer tolerates retry, duplicate delivery, bounded reordering, restart, and partition without double credit under the trusted-warden model.

The central result is deliberately conditional and operational: a site can continue during a partition only from authority already escrowed there. Capacity elsewhere may be stranded. That is the cost of preserving a hard population-wide bound without consensus on every action. The artifact turns the earlier abstract proposal into testable software while retaining honest boundaries around Byzantine faults, restores, migration, external effects, and policy correctness.

A Operation Delta Table

Table 8: Componentwise deltas for Eq. 1. A zero denotes no vector change.

Operation	ΔF	ΔR	ΔC	ΔO	ΔI
Issue root x	$-x$	$+x$	0	0	0
Spawn child x	0	0	0	0	0
Execute cost c	0	$-c$	$+c$	0	0
Close/reclaim r	$+r$	$-r$	0	0	0
Prepare transfer x	$-x$	0	0	$+x$	0
Accept transfer x	$+x$	0	0	0	$+x$
Finalize/checkpoint	0	0	0	0	0

Spawn has zero aggregate ΔR because it subtracts the child allocation from the parent and adds the same vector to the child in one transaction. A duplicate accepted transfer has all-zero deltas because the stored acknowledgement is returned.

B Artifact Reproduction

All Python commands use the repository-local environment:

```
uv sync --all-extras --frozen
uv run --frozen ruff check .
uv run --frozen ruff format --check .
uv run --frozen mypy src
uv run --frozen pytest -m "not e2e" '
-W error::ResourceWarning --cov=lets --cov-report=term-missing
uv run --frozen python deploy/run_acceptance.py
```

The bounded checker is reproduced with:

```
uv run --frozen python -m formal.model_checker
```

The retained mutation record uses `--inject-duplicate-credit-fault`; its expected nonzero exit and minimal counterexample show that duplicate credit violates conservation. The retained TLC record includes the tool version, JAR digest, exact command, configuration/specification digests, queue exhaustion, and state counts. On Windows, the one-command runner is:

```
powershell -NoProfile -File formal/run_tlc.ps1
```

It downloads the pinned TLA+ v1.8.0 JAR into an ignored repository-local directory, verifies its SHA-256 and byte count, and uses the preinstalled Java runtime; it installs no system software.

The post-optimization benchmark suite uses the repository-local Python environment:

```
uv run --frozen python -m benchmarks.run
uv run --frozen python -m benchmarks.profile_scaling
uv run --frozen python -m benchmarks.profile_invariants
uv run --frozen python -m benchmarks.profile_anchor
```

Reviewed host-specific JSON and CSV are retained below benchmarks/baselines; generated profiles and scratch results remain ignored.

The paper build uses a digest-pinned TeX Live container and writes auxiliary files below paper/build:

```
make paper
make paper-check
make paper-render
```

The Docker acceptance runner validates the exact five project-owned volumes it may recreate, starts from clean state, builds the runtime, waits for all health checks and directed fault proxies, writes machine-readable evidence, and stops containers in a finally path. The test-only volumes remain for operator inspection.

C Receipt Verification Checklist

A protected executor should reject unless all of the following hold:

1. the wire record is strict and canonical, with a supported type and no unknown fields;
2. the registered warden/key pair is currently valid at verification time under clock uncertainty;
3. the Ed25519 signature covers the complete receipt payload;
4. every tenant, envelope, configuration-epoch, policy-digest, machine-digest, and warden pin configured by local executor policy matches;
5. the executor audience is exact and the receipt is fresh;
6. the lease sequence strictly advances the durable executor watermark; and
7. the receipt identifier and nonce are claimed once in a filesystem-backed replay store.

Only after this checklist should the executor enter its application-specific effect transaction.

References

- A2A Project. Agent2agent protocol specification, version 1.0. <https://a2a-protocol.org/latest/specification/>, 2026. Official specification; accessed 2026-08-09.
- AGNTCY Contributors. Open agentic schema framework. <https://github.com/agntcy/oasf>, 2026. Canonical project repository; accessed 2026-08-09.
- Isaac Agudo, Carmen Fernandez-Gago, and Javier Lopez. Delegating privileges over finite resources: A quota based delegation approach. In *Formal Aspects in Security and Trust*, volume 5491 of *Lecture Notes in Computer Science*, pages 302–315, 2009. doi: 10.1007/978-3-642-01465-9_20.
- Soufiane Amini, Yassine Benajiba, Cesare Bernardis, Paul Cayet, Hassan Chafi, Abderrahim Fathan, Louis Faucon, Damien Hilloulin, Sungpack Hong, Ingo Kossyk, Tran Minh Son Le, Rhicheck Patra, Sujith Ravi, Jonas Schweizer, Jyotika Singh, Shailender Singh, Weiyi Sun, Kartik Talamadupula, and Jerry Xu. Open agent specification (Agent Spec): A unified representation for AI agents. *arXiv preprint arXiv:2510.04173*, 2025. doi: 10.48550/arXiv.2510.04173.
- Michael P. Andersen, Sam Kumar, Moustafa AbdelBaky, Gabe Fierro, John Kolb, Hyung-Sin Kim, David E. Culler, and Raluca Ada Popa. WAVE: A decentralized authorization framework with transitive delegation. In *28th USENIX Security Symposium*, pages 1375–1392, 2019. URL <https://www.usenix.org/conference/usenixsecurity19/presentation/andersen>.
- ARD Project. Agentic resource discovery specification, version 0.9 draft. <https://github.com/ards-project/ard-spec>, 2026. Canonical draft specification repository; accessed 2026-08-09.
- AstralDeep. LETS: Lineage escrow transition systems. <https://github.com/AstralDeep/LETS>, 2026. Version 0.1.0, Apache-2.0 source artifact.
- Valter Balegas, Diogo Serra, Sergio Duarte, Carla Ferreira, Marc Shapiro, Rodrigo Rodrigues, and Nuno Preguiça. Extending eventually consistent cloud databases for enforcing numeric invariants. In *34th IEEE Symposium on Reliable Distributed Systems*, pages 31–36, 2015. doi: 10.1109/SRDS.2015.32.
- Arnar Birgisson, Joe Gibbs Politz, Ulfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *Network and Distributed System Security Symposium*, 2014. doi: 10.14722/ndss.2014.23212.

- CloudEvents Authors. Cloudevents specification, version 1.0.2. <https://github.com/cloudevents/spec/tree/ce@v1.0.2>, 2022. CNCF specification release; accessed 2026-08-09.
- Amjad Fatmi. Faramesh: A protocol-agnostic execution control plane for autonomous agent systems. *arXiv preprint arXiv:2601.17744*, 2026. doi: 10.48550/arXiv.2601.17744.
- David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987. doi: 10.1016/0167-6423(87)90035-9.
- in-toto Project. in-toto attestation framework specification, version 1.2. <https://github.com/in-toto/attestation/tree/v1.2.0/spec>, 2026. Official specification release; accessed 2026-08-09.
- Anuj Kaul, Qianlong Lan, and Pranay Gupta. Behavioral governance for autonomous AI agents: The agentbound framework. *arXiv preprint arXiv:2606.30970*, 2026. doi: 10.48550/arXiv.2606.30970.
- Leslie Lamport. The temporal logic of actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994. doi: 10.1145/177492.177726.
- Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3C Recommendation, <https://www.w3.org/TR/prov-o/>, 2013. 30 April 2013.
- Model Context Protocol Maintainers. Model context protocol specification, revision 2026-07-28. <https://modelcontextprotocol.io/specification/2026-07-28>, 2026. Official specification; accessed 2026-08-09.
- Patrick E. O’Neil. The escrow transactional method. *ACM Transactions on Database Systems*, 11(4):405–430, 1986. doi: 10.1145/7239.7265.
- Open Container Initiative. OCI image format specification, version 1.1.1. <https://github.com/opencontainers/image-spec/tree/v1.1.1>, 2025. Official specification release; accessed 2026-08-09.
- OWASP Foundation. Cyclonedx specification, version 1.7. <https://cyclonedx.org/specification/overview/>, 2025. Official specification; accessed 2026-08-09.
- Yuxiang Peng and Xiaodi Wu. Stateful governance for concurrent agentic systems. *arXiv preprint arXiv:2608.02764*, 2026. doi: 10.48550/arXiv.2608.02764.
- Anders Rundgren, Ben Jordan, and Samuel Erdtman. RFC 8785: JSON canonicalization scheme (JCS). Internet Engineering Task Force, <https://www.rfc-editor.org/rfc/rfc8785.html>, 2020.
- Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9): 1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- Fred B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. doi: 10.1145/353323.353382.
- Ali Shoker, Paulo Sérgio Almeida, and Carlos Baquero. Exactly-once quantity transfer. In *34th IEEE Symposium on Reliable Distributed Systems Workshops*, pages 68–73, 2015. doi: 10.1109/SRDSW.2015.10.
- SLSA Community. SLSA specification, version 1.2. <https://slsa.dev/spec/v1.2/>, 2025. Approved specification; accessed 2026-08-09.
- SPDX Workgroup. SPDX specification, version 3.0.1. <https://spdx.github.io/spdx-spec/v3.0.1/>, 2024. Linux Foundation specification; accessed 2026-08-09.
- SPIFFE Project. The SPIFFE standard, version 1.15.2. <https://spiffe.io/docs/latest/spiffe-specs/>, 2026. Official standard; accessed 2026-08-09.
- SQLite Project. Write-ahead logging. <https://sqlite.org/wal.html>, 2026. Official documentation; accessed 2026-08-09.
- Qing Ye and Jing Tan. Agent contracts: A formal framework for resource-bounded autonomous AI systems. *arXiv preprint arXiv:2601.08815*, 2026. doi: 10.48550/arXiv.2601.08815.
- Boxuan Zhang, Yi Yu, Jiaxuan Guo, and Jing Shao. Dive into the agent matrix: A realistic evaluation of self-replication risk in LLM agents. *arXiv preprint arXiv:2509.25302*, 2025. doi: 10.48550/arXiv.2509.25302.
- Yingqi Zhang. Agent libos: A runtime substrate for capability-controlled self-evolving LLM agents. *arXiv preprint arXiv:2606.03895*, 2026. doi: 10.48550/arXiv.2606.03895.
- Zhaoxi Zhang and Xiaomei Zhang. Are you still the agent i authorized? earned authority under a fixed ceiling for evolving agents. *arXiv preprint arXiv:2607.23586*, 2026. doi: 10.48550/arXiv.2607.23586.